Trabajo Práctico 2 - Ejercicios extra

SISTEMAS OPERATIVOS

Tracks

1 Señales y control de trabajos (5/10) 3
2 Mini-shell con `fork()` y `exec()` (7/10) 4
3 Carreras y sincronización entre hilos (7/10) 5
4 Cambios de contexto y rendimiento (8/10) 6

Estos tracks amplían la práctica de procesos con problemas pequeños pero cercanos a herramientas reales. Podés empezar por cualquier track. Cada uno tiene una parte principal y una extensión para seguir investigando.

1 Señales y control de trabajos (5/10)

TRACK - 5/10

Descripción: usar señales para pausar, continuar y terminar procesos. Es una base práctica para administrar servidores y depurar programas.

1. Ejecutar `sleep 100` en foreground y detenerlo con `Ctrl-Z`.
2. Consultar trabajo con `jobs`, continuarlo en background con `bg` y traerlo nuevamente con `fg`.
3. Encontrar PID y enviar `SIGSTOP`, `SIGCONT` y `SIGTERM` usando `kill`.
4. Comparar `SIGTERM` con `SIGKILL`. Explicar por qué un programa no puede capturar `SIGKILL`.
5. Escribir programa que instale handler para `SIGINT`, cuente cuántas veces recibió señal y termine limpiamente con segunda.

EXTRA

Investigá diferencia entre señal estándar y señal encolada. Usá `kill -l`, `man 7 signal` y `sigaction()`.

2 Mini-shell con `fork()` y `exec()` (7/10)

TRACK - 7/10

Descripción: construir versión mínima del mecanismo que usa un shell real. Integra parsing, creación de procesos, ejecución y espera.

1. Leer línea usando `fgets()` y separar comando de argumentos.
2. Crear hijo con `fork()` y ejecutar comando con `execvp()`.
3. Hacer que padre espere a hijo con `waitpid()`.
4. Implementar comando interno `cd`, que debe ejecutarse en proceso del shell.
5. Agregar ejecución en background cuando línea termina en `&`.
6. Informar errores de `fork()`, `execvp()` y `waitpid()` con `perror()`.

```c
if (fork() == 0) {
    execvp(args[0], args);
    perror("execvp");
    _exit(127);
}
wait(NULL);
```

BONUS · POR PUNTOS 🪙

Agregá una tubería entre dos comandos usando `pipe()`, `dup2()` y dos procesos hijo. Probá ejecutar `ps aux | grep bash`.

3 Carreras y sincronización entre hilos (7/10)

TRACK - 7/10

Descripción: observar una condición de carrera y corregirla con un mutex. Es una práctica directamente útil para programación concurrente.

1. Crear diez hilos que incrementen contador global un millón de veces cada uno.
2. Ejecutar varias veces y comparar resultado con valor esperado.
3. Explicar por qué `contador++` no es operación indivisible.
4. Proteger incremento con `pthread_mutex_lock()` y `pthread_mutex_unlock()`.
5. Medir tiempo con y sin mutex usando `time`.
6. Confirmar cantidad de hilos con `ps -T`.

EXTRA

Compará mutex con operaciones atómicas usando `stdatomic.h`. Investigá por qué solución correcta puede ser más lenta y cuándo conviene cada alternativa.

4 Cambios de contexto y rendimiento (8/10)

TRACK - 8/10

Descripción: medir qué costo tiene alternar entre procesos e hilos. Conecta modelo teórico del planificador con datos observables.

1. Crear programa que haga trabajo de CPU durante varios segundos.
2. Medir cambios de contexto voluntarios y no voluntarios antes y después usando `/proc/PID/status`.
3. Comparar proceso que usa `sleep()` con otro que ocupa CPU en un loop.
4. Ejecutar dos copias del programa y observar efecto sobre cambios de contexto y tiempo de ejecución.
5. Repetir medición con dos hilos y explicar qué resultados esperás encontrar.
6. Registrar resultados en una tabla y distinguir hipótesis de observación.

BONUS · POR PUNTOS 🪙

Usá `taskset` para fijar afinidad a un núcleo y repetí experimento. Investigá también `getrusage()` y diferencia entre tiempo de usuario y tiempo de sistema.